

HALO

Clean Architecture & Riverpod

Progress Report

Status of the architecture migration on branch sahyog versus current main, as of 1 September 2026.

Item	Value
Product	HALO (Flutter + Firebase)
Repository	classic_1_backup
Architecture branch	sahyog (this is where the work lives)
Current checkout	main (no Riverpod, no features/auth)
Report date	1 September 2026
Scope	Honest inventory — not a launch plan

Read this first

Clean Architecture + Riverpod is not on main. Checking out main (or building from main) shows the old login in lib/main.dart and lib/Bottom Pages/login_page.dart, with Firebase called from widgets. All numbers below for “done” work refer to branch sahyog unless stated otherwise. sahyog is also not merged: main is 1 commit ahead and 29 commits behind origin/main in the working tree snapshot used for this report.

1. Executive summary

The app is not “already on Clean Architecture.” One vertical slice is: authentication and HALO onboarding. That slice has a real domain contract, a Firebase data layer, Riverpod controllers, and a session-based gate. Everything else — home, explore, reels, chat, search, stories, notifications, upload, and the three giant profile pages — still talks to Firebase from UI or singleton services.

Score	Value	How it is measured
Overall app migrated	~10–15%	Dart files using Riverpod (16 of 167) and lines in architecture folders vs ~44,400 total
Auth + onboarding slice	~85–90%	Domain, data, presentation, gate, tests for session rules
Riverpod in the UI	11 of ~87 screens	ConsumerWidget / ConsumerStatefulWidget vs other widget files
Clean feature modules	1 of ~10	Only lib/features/auth exists. No features/feed, chat, reels, profile-write
main branch progress	0%	flutter_riverpod is not a dependency; no lib/features folder
Automated tests	Thin	3 test files; session mapper covered; no Home/Explore/Reels tests

Bottom line: do not rewrite the remaining 85% before a beta. Keep the strangler pattern — finish merging sahyog auth onto a stable branch, then migrate Home / feed / upload the same way, one slice at a time.

2. What “Clean Architecture + Riverpod” means in this project

The intended pattern (already applied to auth) is the standard Flutter feature-first layout, not a textbook enterprise stack with use-case classes for every button.

- **Domain** — pure Dart types and rules. No Firebase imports.
- **Data** — implements the domain contract. Firebase Auth, Firestore, Google, Apple, and phone OTP live only here.
- **Presentation** — pages and widgets. They call Riverpod notifiers. They do not call FirebaseAuth.instance or FirebaseFirestore.instance.
- **Riverpod** — Provider / StreamProvider / StateNotifierProvider. ProviderScope wraps the app in main.dart.
- **Gate** — one Session stream decides Login vs Verify Email vs Category vs Profile form vs Home.

A feature is counted as complete only if all four are true: domain contract, data implementation, Riverpod wiring, and UI free of Firebase.

3. What is complete (branch sahyog)

3.1 Authentication feature module

lib/features/auth is a real three-layer module (~3,150 lines, 16 Dart files). The UI for login, email signup, phone OTP, verify email, and forgot password goes through repositories and Riverpod. New email users get a Firebase account, a verification email, then category + profile. Google / Phone / Apple authenticate first, then the same onboarding if the HALO profile is incomplete. Existing users with onboardingCompleted (or a legacy username + accountType) go to Home.

Layer	Files	Role
Domain	auth_repository.dart, session_mapper.dart, phone_otp_session.dart	Contract: sign in/up, OTP, Google, Apple, phone, verification, password reset, username check, complete onboarding, clear account type
Core session	lib/core/session.dart	SessionStatus: loading, loggedOut, emailVerificationRequired, onboardingRequired, authenticated
Data	firebase_auth_repository.dart (~553 lines)	Only place auth talks to Firebase. Writes users/{uid}. Does not store passwords. Strips secret keys on profile merge
Riverpod	session_controller.dart, phone_auth_controller.dart	authRepositoryProvider, sessionProvider, authActionProvider, onboardingControllerProvider, phoneAuthControllerProvider
Gate	auth_gate.dart, onboarding_gate.dart	Single router from session. Category page if type is null; matching profile form if type is set
Pages	login, email_signup, phone_login, verify_email	Consumer widgets. No Firebase in these pages

Onboarding UI	onboarding_ui.dart + Category + 3 create-account forms	Forms submit via onboardingControllerProvider.completeOnboarding
---------------	--	---

3.2 Auth behaviours that are implemented

- Email/password, username, or mobile identifier login via the repository.
- Email signup creates the account, sends verification, stubs users/{uid} with onboardingCompleted: false.
- Verify Email page: reload, resend, sign out. Password users cannot skip verification.
- Forgot password via sendPasswordResetEmail on the repository.
- Google and Apple sign-in in the data layer (Apple is implemented in code; iOS project config is still incomplete).
- Phone OTP with verification-id refresh; UI pops when session leaves loggedOut.
- Category pick writes accountType only. Back / “Change type” calls clearAccountType so the user is not stuck on Wellness/Guru.
- Profile onboarding (Aspirant / Guru / Wellness) no longer creates a second Firebase account and no longer writes passwords.
- Username uniqueness checked in the data layer.
- Centered toasts (halo_toast) and white/modern onboarding chrome.

3.3 Riverpod providers that exist

Provider	Type	Used by
authRepositoryProvider	Provider<AuthRepository>	All auth controllers
sessionProvider	StreamProvider<Session>	OnboardingGate (app router)
authActionProvider	StateNotifierProvider	Login, signup, verify, Google, Apple, sign-out, password reset
onboardingControllerProvider	StateNotifierProvider	Category + three profile forms
phoneAuthControllerProvider	StateNotifierProvider	Phone OTP page
profileRepositoryProvider	Provider<ProfileRepository>	Profile view (partial)
profileStreamProvider	StreamProvider	Profile view (partial)
unifiedProfileDataProvider	StreamProvider.family	Dynamic profile page (partial)

3.4 Tests that exist for this slice

test/profile_type_test.dart covers session mapping: unverified email is gated; missing HALO doc goes to category selection; onboardingCompleted or legacy username+accountType goes to authenticated; Google/phone users are not forced through email verification. test/profile_widgets_test.dart and test/widget_test.dart are small smoke tests. There are no integration tests for login, OTP, or Home.

4. What is only partial

4.1 Profile viewing (lib/screens/profile)

A modular profile UI was started: 44 files, ~2,380 lines, with ProfileKind, a thin repository barrel, and a Riverpod stream for ProfileData. That is folder structure plus some extraction — not a finished Clean Architecture feature. Several profile widgets still query Firestore directly (recent posts, online dot). The old Profile Pages were not deleted.

Old profile page (still in repo)	Approx. lines	Riverpod / domain
lib/Profile Pages/aspirant_profile_page.dart	3,351	No
lib/Profile Pages/guru_profile_page.dart	5,522	No
lib/Profile Pages/wellness_profile_page.dart	4,188	No
lib/Profile Pages/edit_profile_sections.dart	Firestore × 18 call sites	No

Those three pages alone are ~13,000 lines of AI-generated UI. They are the largest remaining refactor, and they should not be rewritten before Aspirant beta testers are using the app.

4.2 Onboarding profile forms

createaspirantaccount.dart (~782 lines), createguruaccount.dart (~953), createwellnessaccount.dart (~830) now submit through Riverpod and no longer call createUserWithEmailAndPassword. They are still oversized form widgets, not domain models. Guru/Wellness certification fields can still store local file paths. That is a data-quality bug, not an architecture win.

4.3 Services folder

lib/services has 34 Dart files / ~3,300 lines (feed, explore, reels, stories, upload, follow, save, search, video pipeline). These are singleton helpers, not domain repositories, and the screens that call them are not Riverpod consumers. This is a useful extraction for ranking and video memory — it is not Clean Architecture.

5. What has not been migrated

On sahyog, about 54 Dart files still reference FirebaseAuth.instance or FirebaseFirestore.instance. The high-impact screens:

Area	Main files	Firebase in UI	Riverpod	Clean feature folder
Home / feed	HomePage.dart (~1,743 lines)	17 call sites	No	No
Explore	ExplorePage.dart (~1,522 lines)	9	No	No
Reels	reels_feed.dart (~524 lines)	7	No	No
Chat	chat_screen / chat_list / user_list	12+	No	No
Search	SearchPage.dart	7	No	No
Notifications	NotificationPage.dart	4	No	No
Stories	story_viewer, user_story_pager	7	No	No
Upload / posts	AddPostPage, upload_service	Yes	No	No

Settings / privacy	SettingsPage, PrivacySettingsPage	4	No	No
Interest selection	interest_selection_page.dart	2	No	No
Duplicate home/login	home_page.dart, leftover startup_router	Yes	Leftover	No

5.1 main branch (current checkout)

The workspace on 1 September 2026 is on main. That tree has 74 Dart files, no flutter_riverpod dependency, no lib/features, and login still mixed into lib/main.dart. Building an APK from main does not include the architecture work described in sections 3–4. Merge or cherry-pick sahyog before treating auth as “done” on the branch you ship.

6. Feature scorecard

Feature	Domain	Data repo	Riverpod	UI free of Firebase	Tests	Verdict
Email / Google / Phone / Apple login	Yes	Yes	Yes	Yes (login pages)	Session mapper	Done
Email verification gate	Yes	Yes	Yes	Yes	Yes	Done
Password reset	Yes	Yes	Yes	Mostly	No	Done
Session routing / AuthGate	Yes	Yes	Yes	Yes	Yes	Done
Category pick + back/clear type	Yes	Yes	Yes	Yes	Partial	Done
Aspirant / Guru / Wellness onboarding save	Partial	Yes	Yes	Mostly	No	Partial
Profile view (new screens/profile)	Thin	Thin	Partial	No	Smoke	Partial
Legacy profile pages	No	No	No	No	No	Not started
Home / feed	No	Service only	No	No	No	Not started
Explore / search ranking	No	Service only	No	No	No	Not started
Reels / video pipeline	No	Services	No	No	No	Not started
Stories	No	Service only	No	No	No	Not started
Chat	No	chat_service	No	No	No	Not started
Upload / Cloudinary	No	Services	No	No	No	Not started
Notifications	No	No	No	No	No	Not started
iOS Sign in with Apple + plist	Code exists	Code exists	Yes	N/A	No	Partial

7. Size inventory (sahyog)

Bucket	Files	Approx. lines	Share of lib/
--------	-------	---------------	---------------

All Dart under lib/	167	44,393	100%
features/auth (Clean + Riverpod)	16	3,148	7%
core/ (session, theme, toast, splash)	4	379	<1%
screens/profile (modular UI, mixed)	44	2,380	5%
services/ (singletons, not Riverpod UI)	34	3,308	7%
Legacy Profile Pages	4	~13,060	~29%
Home + Explore	2	~3,265	~7%
Riverpod imports anywhere	16 files	—	10% of files
Consumer widgets	11 files	—	~13% of UI files

Architecture folders look larger than the real migration. screens/profile and services/ inflate the “structured code” count. The only slice that meets the definition in section 2 is auth.

8. Known leftovers and risks

- **startup_router.dart** is leftover and still touches Firebase. Interest selection is no longer an auth gate.
- Duplicate HomePage.dart vs home_page.dart and old login paths can confuse merges.
- Guru/Wellness certification images may store local paths instead of uploaded URLs.
- Email verification depends on Firebase (noreply@halo-fb212.firebaseio.com). Spam and rate limits are product issues, not architecture gaps.
- iOS: GoogleService-Info.plist / Apple Sign-In capability / display name were incomplete on main. Repository methods exist on sahyog; store listing and signing are not done.
- sahyog unmerged. Shipping from main silently drops the entire auth rewrite.
- No crash/jank budget, no CI test gate beyond the three local tests.

9. Recommended next slices (do not rewrite the whole app)

Use the same pattern as auth. One feature folder, one repository, a few providers, then delete Firebase from that screen.

Order	Slice	Why this next	Difficulty
0	Merge sahyog auth onto the branch you ship; smoke-test login on a device	Otherwise the work in this report is not in the APK	Medium (merge conflict risk)
1	Sign-out / session on Home and Settings (stop FirebaseAuth.instance there)	Auth is already the source of truth; Home still bypasses it	Low
2	Feed / Home read path → features/feed	Highest user time, 17 Firebase call sites	High
3	Upload / new post → features/upload	Crashes and duplicate posts hurt beta more than profile chrome	High
4	Aspirant profile view + edit only	Needed for Aspirant-only beta; leave Guru/Wellness giants	Medium
5	Chat list + thread	Direct Firebase in several chat files	Medium

6	Explore / reels / stories	Already have ranking services; wrap them, do not rewrite ranking	High
7	Guru then Wellness profile pages	Only after Aspirant beta is stable	High
8	iOS TestFlight (plist, Apple Sign-In, signing)	Not a “minor toggle”	Medium–High

9.1 Definition of done for the next slice

- `lib/features/<name>/{domain,data,presentation}` exists.
- Widgets in that slice do not import `firebase_auth` or `cloud_firestore`.
- At least one unit test for the mapper or repository contract.
- Old duplicate file deleted or reduced to a re-export.

10. main vs sahyog at a glance

Topic	main (current checkout)	sahyog (architecture work)
Dart files in <code>lib/</code>	74	167
<code>flutter_riverpod</code>	Not in pubspec	^2.6.1
<code>ProviderScope</code>	No	Yes, in <code>main.dart</code>
<code>lib/features/auth</code>	Missing	16 files, 3 layers
Session gate	Ad-hoc in <code>main / login_page</code>	<code>OnboardingGate</code> + <code>SessionStatus</code>
Login Firebase in widgets	Yes	No (login pages)
Onboarding creates 2nd account	Yes (legacy forms)	No — <code>completeProfileOnboarding</code> on same uid
Profile module	Monolith pages only	New screens/profile + old pages both present
Tests	<code>widget_test</code> smoke	session mapper + profile smoke

11. Conclusion

Clean Architecture and Riverpod are started, not finished. Auth is the proof that the pattern works: domain contract, Firebase isolated in data, Riverpod controllers, and a single session gate. That is roughly one-tenth of the app by files, and the most important tenth for a correct login.

Calling the project “architected” would be inaccurate while Home, Explore, Reels, Chat, and ~13,000 lines of profile UI still call Firebase. The fastest path to a beta is to protect the auth slice, merge it, and repeat the same pattern on feed and upload — not to pause launch for a full rewrite.

Appendix A — Auth file list (sahyog)

- `lib/core/session.dart`
- `lib/core/halo_toast.dart`
- `lib/core/halo_theme.dart`
- `lib/core/halo_splash.dart`
- `lib/features/auth/domain/auth_repository.dart`

- lib/features/auth/domain/session_mapper.dart
- lib/features/auth/domain/phone_otp_session.dart
- lib/features/auth/data/firebase_auth_repository.dart
- lib/features/auth/presentation/session_controller.dart
- lib/features/auth/presentation/phone_auth_controller.dart
- lib/features/auth/presentation/auth_gate.dart
- lib/features/auth/presentation/onboarding_gate.dart
- lib/features/auth/presentation/onboarding_ui.dart
- lib/features/auth/presentation/pages/login_page.dart
- lib/features/auth/presentation/pages/email_signup_page.dart
- lib/features/auth/presentation/pages/phone_login_page.dart
- lib/features/auth/presentation/pages/verify_email_page.dart
- lib/features/auth/presentation/widgets/login_button.dart
- lib/features/auth/presentation/widgets/legal_dialogs.dart
- lib/Category/categorypage.dart
- lib/Category/createaspirantaccount.dart
- lib/Category/createguruaccount.dart
- lib/Category/createwellnessaccount.dart
- test/profile_type_test.dart

Appendix B — Method notes

Counts were taken from git show against branch sahyog on 1 September 2026 (PowerShell Measure-Object -Line on each Dart file). “Call sites” are git grep counts of FirebaseAuth.instance and Firestore.instance. Percentages are rounded. This document does not include a time/cost estimate; those live in the separate beta roadmap discussion.